

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к практической работе №3
на тему:
«Мультизадачность в защищенном режиме»
БГУИР 6-05-0612-02 125

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

(дата, подпись студента)

Проверил ассистент каф. информатики
КАЛИНОВСКАЯ Анастасия Александровна

(дата, подпись преподавателя)

Минск 2025

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Написать программу, реализующую мультизадачность в защищенном режиме. Программа должна переключить процессор в защищенный режим, а затем запустить на выполнение 2-3 задачи.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В рамках лабораторной работы была разработана система многозадачности для архитектуры x86, состоящая из загрузчика (boot.asm) и 32-битного ядра (kernel32.asm). Программа демонстрирует циклическое планирование задач на основе прерываний таймера.

```
; boot.asm – 16-bit boot sector: LBA load → Protected Mode
bits 16
org 0x7C00

start:
    cli
    xor ax, ax
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov sp, 0x7C00

    mov [BootDrive], dl

    ; Enable A20
    in  al, 0x92
    or  al, 0000_0010b
    out 0x92, al

#include "kernel_sectors.inc"

; ----- Load kernel (INT 13h extensions, LBA) to 0x0010_0000 -----
--
    mov dl, [BootDrive]
    mov ax, 0x1000
    mov es, ax
    xor bx, bx

    mov ax, KERNEL_SECTORS
    mov [DAP+2], ax
    mov word [DAP+4], 0x0000
    mov word [DAP+6], 0x1000
    mov dword [DAP+8], 1
    mov dword [DAP+12], 0

    mov si, DAP
    mov ah, 0x42
```

```

    int 0x13
    jc  disk_error

; ----- Enter Protected Mode -----
    lgdt [gdt_desc]
    cli
    mov eax, cr0
    or  eax, 1
    mov cr0, eax
    jmp dword 0x08:0x0010000    ; IMPORTANT: 32-bit far jump

disk_error:
    mov si, msgDE
    call print
.hang:
    hlt
    jmp .hang

print:
    pusha
    mov bx, 0xB800
    mov es, bx
    mov di, 0
.next:
    lodsb
    or al, al
    jz .done
    stosb
    mov al, 0x4F
    stosb
    jmp .next
.done:
    popa
    ret

msgDE db 'DE',0

; ----- Data & GDT -----
align 4
BootDrive db 0

DAP:
    db 16
    db 0
    dw 0
    dw 0
    dw 0
    dd 0
    dd 0

align 8
gdt_start:
    dq 0x0000000000000000
    dq 0x00CF9A000000FFFF

```

```

        dq 0x00CF92000000FFFF
gdt_end:

gdt_desc:
        dw gdt_end - gdt_start - 1
        dd gdt_start

times 510-($-$$) db 0
dw 0xAA55

; kernel32.asm - 32-bit flat kernel @ phys 0x0010_0000
bits 32
org 0x00100000

%define KERNEL_CS 0x08
%define KERNEL_DS 0x10
%define VGA_MEM 0xB8000
%define ROWS 25
%define COLS 80

%define PIT_HZ 100
%define PIT_DIVISOR 11932 ; 1193182 / 100 ≈ 11932

%define TASKS 3

; -----
; Entry
; -----
start:
        mov ax, KERNEL_DS
        mov ds, ax
        mov es, ax
        mov fs, ax
        mov gs, ax
        mov ss, ax
        mov esp, kernel_stack_top

        call cls
        mov esi, banner
        mov edi, (0*COLS + 0)*2
        call kprint_at

        mov esi, banner2
        mov edi, (1*COLS + 0)*2
        call kprint_at

; IDT, PIC, PIT, tasks
call idt_init
call pic_remap
mov al, 0b11111110 ; unmask only IRQ0
out 0x21, al
mov al, 0xFF

```

```

    out 0xA1, al

    call pit_init
    call setup_tasks
    sti

.halt:
    hlt
    jmp .halt

; -----
; Text helpers
; -----
cls:
    pushad
    mov edi, VGA_MEM
    mov ecx, ROWS*COLS
    mov ax, 0x0720
.fill:
    stosw
    loop .fill
    popad
    ret

; print zero-terminated string at VGA offset EDI (cells*2)
kprint_at:
    pushad
    mov ebx, VGA_MEM
    add edi, ebx
.next:
    lodsb
    test al, al
    jz .done
    mov ah, 0x0F
    stosw
    jmp .next
.done:
    popad
    ret

; putc_xy: AL=char, BH=attr, DL=x, DH=y    (keeps AL intact)
putc_xy:
    pushad
    movzx ecx, dl
    movzx edx, dh
    imul edx, COLS
    add ecx, edx
    lea edi, [VGA_MEM + ecx*2]
    mov byte [edi], al
    mov byte [edi+1], bh
    popad
    ret

; clear_row: erase row DH with spaces (attr 0x07)

```

```

clear_row:
    pushad
    movzx eax, dh
    imul eax, eax, COLS
    lea edi, [VGA_MEM + eax*2]
    mov ecx, COLS
    mov ax, 0x0720
    rep stosw
    popad
    ret

; -----
; PIC / PIT
; -----

io_wait:
    out 0x80, al
    ret

pic_remap:
    pushad
    mov al, 0x11
    out 0x20, al
    call io_wait
    out 0xA0, al
    call io_wait

    mov al, 0x20        ; master offset
    out 0x21, al
    call io_wait
    mov al, 0x28        ; slave offset
    out 0xA1, al
    call io_wait

    mov al, 0x04        ; Master: Slave at IRQ2
    out 0x21, al
    call io_wait
    mov al, 0x02        ; Slave identity
    out 0xA1, al
    call io_wait

    mov al, 0x01        ; 8086 mode
    out 0x21, al
    call io_wait
    out 0xA1, al
    call io_wait
    popad
    ret

pit_init:
    pushad
    mov al, 0x36
    out 0x43, al
    mov ax, PIT_DIVISOR
    out 0x40, al        ; low

```

```

    mov al, ah
    out 0x40, al        ; high
    popad
    ret

; -----
; Scheduler state
; -----
align 16
current    dd -1
tcb_esps:  times TASKS dd 0
x0 dd 0
x1 dd 0
x2 dd 0
spin_idx db 0
spin_tab db '|','/','-','\\'

; -----
; Timer ISR (IRQ0) - simple RR + spinner
; -----
timer_isr:
    cli
    pusha

    mov eax, [current]
    cmp eax, -1
    je .first

    mov ebx, eax
    shl ebx, 2
    mov [tcb_esps + ebx], esp

    inc eax
    cmp eax, TASKS
    jl .set
    xor eax, eax
.set:
    mov [current], eax
    shl eax, 2
    mov esp, [tcb_esps + eax]
    jmp .spin

.first:
    mov eax, 0
    mov [current], eax
    shl eax, 2
    mov esp, [tcb_esps + eax]

.spin:
    ; spinner in top-right corner (79,0)
    mov bl, [spin_idx]
    movzx eax, bl
    mov al, [spin_tab + eax]
    mov bh, 0x0F

```

```

    mov dl, 79
    mov dh, 0
    call putc_xy
    inc bl
    and bl, 3
    mov [spin_idx], bl

.eoi:
    mov al, 0x20
    out 0x20, al
    popa
    iret

; -----
; IDT - zeroed table, fill gate 32 at runtime
; -----
align 8
IDT:
    times (256*8) db 0
IDT_end:

IDT_descriptor:
    dw IDT_end - IDT - 1
    dd IDT

idt_init:
    pushad
    mov eax, timer_isr
    mov ebx, 32*8
    mov edx, IDT
    mov [edx + ebx + 0], ax          ; offset low
    mov word [edx + ebx + 2], KERNEL_CS
    mov byte [edx + ebx + 4], 0
    mov byte [edx + ebx + 5], 0x8E
    shr eax, 16
    mov [edx + ebx + 6], ax          ; offset high
    lea eax, [IDT_descriptor]
    lidt [eax]
    popad
    ret

; -----
; Tasks - leave trail; clear row on wrap-around
; -----
setup_tasks:
    pushad
    mov edi, task0_stack_top
    mov ebx, task0
    call build_initial_stack
    mov [tcb_esps + 0*4], edi

    mov edi, task1_stack_top
    mov ebx, task1
    call build_initial_stack

```



```

    mov [tcb_esps + 1*4], edi

    mov edi, task2_stack_top
    mov ebx, task2
    call build_initial_stack
    mov [tcb_esps + 2*4], edi
    popad
    ret

; IN: EDI=stack top, EBX=entry EIP
build_initial_stack:
    sub edi, 4
    mov dword [edi], 0x00000202      ; IF=1
    sub edi, 4
    mov dword [edi], KERNEL_CS
    sub edi, 4
    mov dword [edi], ebx
    mov ecx, 8
.fill:
    sub edi, 4
    mov dword [edi], 0
    loop .fill
    ret

; --- tasks (draw only; on wrap -> clear row and continue) ---
task0:
.loop0:
    mov bh, 0x0F
    mov dl, byte [x0]
    mov dh, 5
    mov al, 'A'
    call putc_xy

    mov eax, [x0]
    inc eax
    cmp eax, COLS
    jb .ok0
    xor eax, eax
    mov dh, 5
    call clear_row
.ok0:
    mov [x0], eax

    call short_delay
    jmp .loop0

task1:
.loop1:
    mov bh, 0x0F
    mov dl, byte [x1]
    mov dh, 7
    mov al, 'B'
    call putc_xy

```

```

        mov eax, [x1]
        inc eax
        cmp eax, COLS
        jb .ok1
        xor eax, eax
        mov dh, 7
        call clear_row
.ok1:
        mov [x1], eax

        call short_delay
        jmp .loop1

task2:
.loop2:
        mov bh, 0x0F
        mov dl, byte [x2]
        mov dh, 9
        mov al, 'C'
        call putc_xy

        mov eax, [x2]
        inc eax
        cmp eax, COLS
        jb .ok2
        xor eax, eax
        mov dh, 9
        call clear_row
.ok2:
        mov [x2], eax

        call short_delay
        jmp .loop2

short_delay:
        push ecx
        mov ecx, 200000
.slp:
        loop .slp
        pop ecx
        ret

; -----
; Stacks
; -----
align 16
kernel_stack: times 4096 db 0
kernel_stack_top:

align 16
task0_stack: times 4096 db 0
task0_stack_top:

align 16

```

```

task1_stack:  times 4096 db 0
task1_stack_top:

align 16
task2_stack:  times 4096 db 0
task2_stack_top:

; -----
; Strings
; -----
banner  db 'Protected-mode multitasking demo (IRQ0 round-robin)',0
banner2 db 'Tasks: A (row 5), B (row 7), C (row 9). Press Ctrl+C to quit QEMU.',0

```

ВЫВОД

В рамках лабораторной работы была успешно реализована система вытесняющей многозадачности в защищённом режиме x86 с использованием переключения контекста по прерыванию таймера. Представленное решение наглядно демонстрирует эффективность использования аппаратных прерываний и механизмов защищённого режима для организации циклического планирования задач, позволяя параллельно выполнять несколько процессов и тем самым повышая общую эффективность использования ресурсов процессора.